THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

OBJECT ORIENTED PROGRAMMING

ASSIGNMENT NO.3

SUBMITTED BY: **RIDHIKA BANSAL**
1024150280
2O31

1.Write a program to implement (a) pointer to an object (b) this pointer. Practice both '.' (dot operator) and '→' (arrow operator).

```
#include <iostream>
using namespace std;
class Rectangle {
private:
    int length;
    int width;
public:
    Rectangle(int l = 0, int w = 0) {
        length = l;
        width = w;
    }
    void setDimensions(int length, int width) {
        this->length = length;
        this->width = width;
    }
    int getArea() {
        return length * width;
    }

    Rectangle& compareAndReturnLarger(Rectangle& other) {
        if (this->getArea() > other.getArea()) {
            return *this;
        } else {
```

```
        return other;
    }
  }
  void display() {
    cout << "Length: " << length << ", Width: " << width << endl;
  }
};
int main() {
  cout << "--- (a) Pointer to an Object & Operator Practice ---" << endl;
  Rectangle rect1;
  Rectangle* ptr = &rect1;

  ptr->setDimensions(10, 5);
  cout << "Area using ptr->getArea(): " << ptr->getArea() << endl;

  (*ptr).setDimensions(8, 4);
  cout << "Area using (*ptr).getArea(): " << (*ptr).getArea() << endl;
  rect1.setDimensions(12, 6);
  cout << "Area using rect1.getArea(): " << rect1.getArea() << endl;
  cout << "\n--- (b) 'this' Pointer Practice ---" << endl;
  Rectangle rect2(15, 10);
  cout << "Rectangle 1 - ";
  rect1.display();
  cout << "Rectangle 2 - ";
  rect2.display();
```

```
    Rectangle largerRect = rect1.compareAndReturnLarger(rect2);
    cout << "\nThe larger rectangle has dimensions: ";
    largerRect.display();
    return 0;
}
```

Output

```
--- (a) Pointer to an Object & Operator Practice ---
Area using ptr->getArea(): 50
Area using (*ptr).getArea(): 32
Area using rect1.getArea(): 72

--- (b) 'this' Pointer Practice ---
Rectangle 1 - Length: 12, Width: 6
Rectangle 2 - Length: 15, Width: 10

The larger rectangle has dimensions: Length: 15, Width: 10


=== Code Execution Successful ===
```

2.Write a program to swap private values of two classes using a friend function.

```
#include <iostream>
using namespace std;

class ClassB;

class ClassA {
private:
    int valueA;

public:
    ClassA(int val) {
        valueA = val;
    }

    void display() {
        cout << "ClassA value: " << valueA << endl;
```

```
    }

    friend void swapValues(ClassA& a, ClassB& b);
};

class ClassB {
private:
    int valueB;

public:
    ClassB(int val) {
        valueB = val;
    }

    void display() {
        cout << "ClassB value: " << valueB << endl;
    }

    friend void swapValues(ClassA& a, ClassB& b);
};

void swapValues(ClassA& a, ClassB& b) {
    int temp = a.valueA;
    a.valueA = b.valueB;
    b.valueB = temp;
}

int main() {
    ClassA objA(10);
    ClassB objB(20);

    cout << "Before Swapping:" << endl;
    objA.display();
    objB.display();

    swapValues(objA, objB);

    cout << "\nAfter Swapping:" << endl;
    objA.display();
    objB.display();

    return 0;
}
```

```
Output
Before Swapping:
ClassA value: 10
ClassB value: 20

After Swapping:
ClassA value: 20
ClassB value: 10


=== Code Execution Successful ===
```

3.Write a program to add data objects of two different classes using friend functions.

```
#include <iostream>
using namespace std;
```

```
class ClassB;
class ClassA {
private:
   int valueA;
public:
   ClassA(int val) {
      valueA = val;
   }
   void display() {
      cout << "ClassA value: " << valueA << endl;
   }

   friend int addValues(ClassA a, ClassB b);
};

class ClassB {
private:
   int valueB;

public:
   ClassB(int val) {
      valueB = val;
   }

   void display() {
      cout << "ClassB value: " << valueB << endl;
   }

   friend int addValues(ClassA a, ClassB b);
};

int addValues(ClassA a, ClassB b) {
   return a.valueA + b.valueB;
}

int main() {
   ClassA objA(25);
   ClassB objB(50);

   objA.display();
   objB.display();

   int sum = addValues(objA, objB);

   cout << "\nSum of private values: " << sum << endl;
   return 0;
}
```

Output

```
ClassA value: 25
ClassB value: 50

Sum of private values: 75

=== Code Execution Successful ===
```

4.Write a program to demonstrate the working of friend class.

```
#include <iostream>
using namespace std;
class BankAccount {
private:
    string accountHolder;
    double balance;
public:
    BankAccount(string name, double initialBalance) {
        accountHolder = name;
        balance = initialBalance;
    }
    friend class Auditor;
};

class Auditor {
public:
    void inspectAccount(BankAccount& account) {
        cout << "--- Audit Report ---" << endl;
        cout << "Account Holder: " << account.accountHolder << endl;
        cout << "Current Balance: $" << account.balance << endl;

        if (account.balance < 0) {
            cout << "Warning: Account is overdrawn!" << endl;
        } else {
            cout << "Status: Account is in good standing." << endl;
        }
    }
};

int main() {
    BankAccount customer1("Alice Smith", 1500.50);
    BankAccount customer2("Bob Jones", -250.00);
    Auditor bankAuditor;
    bankAuditor.inspectAccount(customer1);
    cout << endl;
    bankAuditor.inspectAccount(customer2);
    return 0;
}
```

Output

```
--- Audit Report ---
Account Holder: Alice Smith
Current Balance: $1500.5
Status: Account is in good standing.

--- Audit Report ---
Account Holder: Bob Jones
Current Balance: $-250
Warning: Account is overdrawn!


=== Code Execution Successful ===
```

5.Write a program using Array of Objects to display area of multiple rectangles.

```
#include <iostream>
using namespace std;
class Rectangle {
private:
  int length;
  int width;
public:
  void setDimensions(int l, int w) {
    length = l;
    width = w;
  }
  int getArea() {
    return length * width;
  }
};
int main() {
  Rectangle rects[3];
  int l, w;
  for (int i = 0; i < 3; i++) {
    cout << "Enter length and width for Rectangle " << (i + 1) << ": ";
    cin >> l >> w;
    rects[i].setDimensions(l, w);
  }
  cout << "\n--- Areas of Rectangles ---" << endl;
  for (int i = 0; i < 3; i++) {
    cout << "Rectangle " << (i + 1) << " Area: " << rects[i].getArea() << endl;
  }
  return 0;
}
```

Output

```
Enter length and width for Rectangle 1: 4 5
Enter length and width for Rectangle 2: 7 8
Enter length and width for Rectangle 3: 9 6

--- Areas of Rectangles ---
Rectangle 1 Area: 20
Rectangle 2 Area: 56
Rectangle 3 Area: 54


=== Code Execution Successful ===
```

6.Write a program to define function *cube()* as inline for calculating cube of a number.

```
#include <iostream>
using namespace std;
inline double cube(double num) {
return num * num * num;
```

```
}

int main() {

   double number;

   cout << "Enter a number: ";

   cin >> number;

   cout << "The cube of " << number << " is: " << cube(number) << endl;

   return 0;

}
```

Output

```
Enter a number: 22
The cube of 22 is: 10648


=== Code Execution Successful ===
```

7.Write a program to pass an object as an argument and return the object from a function.

a. Use pass-by-value
b. Use pass-by reference

```
#include <iostream>
using namespace std;

class Number {
private:
   int value;

public:
   Number(int v = 0) {
      value = v;
   }

   void setValue(int v) {
      value = v;
   }

   int getValue() {
      return value;
   }

   void display(string name) {
      cout << name << " value: " << value << endl;
   }
};
```

```
Number modifyByValue(Number num) {
   num.setValue(num.getValue() + 10);
   return num;
}

Number& modifyByReference(Number& num) {
   num.setValue(num.getValue() + 20);
   return num;
}

int main() {
   cout << "--- (a) Pass-by-Value ---" << endl;
   Number obj1(50);
   obj1.display("Original obj1");

   Number result1 = modifyByValue(obj1);

   result1.display("Returned result1");
   obj1.display("Original obj1 after function call");

   cout << "\n--- (b) Pass-by-Reference ---" << endl;
   Number obj2(50);
   obj2.display("Original obj2");

   Number& result2 = modifyByReference(obj2);

   result2.display("Returned result2");
   obj2.display("Original obj2 after function call");

   return 0;
}
```

Output

```
--- (a) Pass-by-Value ---
Original obj1 value: 50
Returned result1 value: 60
Original obj1 after function call value: 50

--- (b) Pass-by-Reference ---
Original obj2 value: 50
Returned result2 value: 70
Original obj2 after function call value: 70


=== Code Execution Successful ===
```